

Compositional Latent Space

Suresh Murugaiyan

July 2026

1 Introduction

We train a *compositional autoencoder* (Path A of the compositional operator-network framework) on the FlowBench 2D lid-driven cavity dataset: steady Navier–Stokes flow with an object placed inside the cavity. The autoencoder compresses each steady flow field into a small latent vector $\mathbf{z}$ and reconstructs the field from it — a nonlinear analogue of POD. The key structural choice is that $\mathbf{z}$ is not one opaque vector; it is partitioned by architecture into three named blocks, each intended to carry one physical factor:

$$\mathbf{z} = [\underbrace{\mathbf{z}_\mu}_{\text{regime (4)}} \parallel \underbrace{\mathbf{z}_g}_{\text{geometry (32)}} \parallel \underbrace{\mathbf{z}_\xi}_{\text{residual (16)}}] \in \mathbb{R}^{52}. \quad (1)$$

The hypothesis under test: after training, $\mathbf{z}_\mu$ should contain the Reynolds number, $\mathbf{z}_g$ should contain the object shape, and neither should leak into the other. Because the flow is steady, there is no dynamics block $\mathbf{z}_\eta$ and no latent propagator Φ in this first version; those enter later with time-dependent cases.

2 Dataset

Each FlowBench sample is a steady lid-driven cavity solution at 512×512 resolution. The data loader (`data/dataset.py`) unpacks the raw `.npz` arrays into:

- **fields** — the (u, v, p) solution, resized $512^2 \rightarrow 256^2$ to keep training cheap;
- **sdf** — the signed distance field of the object (the geometry label);
- **mask** — binary fluid mask (1 in fluid, 0 in the solid);
- **log_re** — the regime label. Re is stored as a constant channel, so a single pixel is extracted, transformed to $\log_{10} \text{Re}$ (Re spans orders of magnitude), and standardized with *train-set* statistics; the test set reuses those statistics so both splits share one scale.

The loader also computes three cheap per-sample geometry summaries from the mask (solid area fraction, centroid x , centroid y). These are never used during training; they serve only as ground-truth targets for the linear-probe diagnostic of Section 5.

3 Network architecture

Four modules (`models/compositional/networks.py`):

- **FieldEncoder** $\mathcal{E}$ — a CNN that halves the spatial resolution repeatedly ($256 \rightarrow 8$), pools, and terminates in *three separate linear heads* producing $\mathbf{z}_\mu, \mathbf{z}_g, \mathbf{z}_\xi$. The blocks are architecturally separate from the start.
- **FieldDecoder** $\mathcal{D}$ — the mirror image: the concatenated $\mathbf{z}$ is reshaped to an 8×8 feature map and upsampled back to a 256^2 (u, v, p) field.
- **RegimeHead** — a small MLP that reads $\mathbf{z}_\mu$ *only* and must predict $\log \text{Re}$.

- **SDFHead** — a small decoder that reads z_g *only* and must redraw the object’s SDF at 64×64 .

The two heads are the supervision mechanism: since the regime head sees only z_μ , the encoder is forced to route Reynolds-number information into that block, and likewise shape information into z_g .

4 Training objective

Each optimization step combines four losses (`models/compositional/compositional_ae.py`), weighted by λ -coefficients set in the configuration file:

1. **Masked reconstruction (L1).** MSE between decoded and true fields, evaluated over fluid points only: predictions inside the solid are zeroed and the error is normalized by the fluid node count, so samples with large objects are not unfairly weighted.
2. **Regime supervision.** MSE of RegimeHead(z_μ) against the standardized $\log_{10}$ Re. Pulls Re into z_μ .
3. **Geometry supervision.** MSE of SDFHead(z_g) against the downsampled true SDF. Pulls shape into z_g .
4. **Cross-block decorrelation (L6).** The mean absolute Pearson correlation, computed across the batch, between every coordinate of one block and every coordinate of another, penalized for all three block pairs. Losses 2–3 pull information *into* the correct block; this term pushes redundant copies *out of* the wrong blocks.

The residual block z_ξ receives no supervision at all: it is free capacity that absorbs whatever the reconstruction needs beyond (Re, shape), which is exactly the role of the residual block in the framework.

Training is driven by `main.py`: it loads the YAML configuration, seeds all libraries, builds the train/test datasets (passing the train Re-statistics to the test set), and trains with PyTorch Lightning, checkpointing on the best validation reconstruction error.

5 Linear-probe diagnostics

The diagnostic (`diagnostics/probes.py`) is the actual scientific test. After training, the entire test set is encoded, and for each (block, target) pair a ridge regression is fitted with 5-fold cross-validation; the resulting R^2 values are tabulated. For a compositional latent we expect the block-diagonal pattern of Table 1.

Table 1: Expected linear-probe R^2 pattern for a compositional latent space. High R^2 on the matching block and low R^2 elsewhere indicates a successful factorization; a high off-diagonal entry indicates leakage.

Target	z_μ	z_g	z_ξ
log Re	high	low	low
Area fraction	low	high	low
Centroid x	low	high	low
Centroid y	low	high	low

If the trained latent reproduces this pattern, the factorization worked: Re is linearly readable from the regime block and *not* from the geometry block, and vice versa. A high R^2 in an off-diagonal cell means information leaked — exactly what the planned loss ablations are designed to study (e.g., does removing the decorrelation penalty cause leakage?).

6 Next steps

Deliberately excluded from this first version and planned as increments once the baseline trains and the probe table looks sane:

- swap-consistency loss (L12): decode $\mathbf{z}_\mu$ from one sample with $\mathbf{z}_g$ from another and compare against the true cross-combination;
- group-structured minibatches for iVAE-style identifiability;
- concept-vector arithmetic diagnostic;
- INR-style implicit decoder; HSIC (nonlinear) decorrelation;
- dynamics block $\mathbf{z}_\eta$ and latent propagator Φ for time-dependent cases.

A Appendix: Running the code on Nova cluster

- STEP1: Login into the Nova cluster `ssh username@nova.its.iastate.edu`
- STEP2: Activate the virtual environment that contains all the required packages.
- STEP3: Request resource allocation from Nova using `salloc -N 1 -n 4 -t 06:00:00 --gres=gpu:a100 --mem 369G`